Lab 3 – Complete Report

1. Question / Problem Statement

Part 1 – Performance Analysis of Greedy Algorithms: Implement three greedy algorithms – Fractional Knapsack, Activity Selection and Huffman Coding – and measure their execution time and memory usage for input sizes 100, 500, 1000 and 5000. Compare the experimental results with the theoretical time and space complexities.

Part 2 – Visualizations and Performance Graphs: Visualize the output of each algorithm (resource-allocation chart, activity timeline, Huffman tree) and plot execution-time and memory-usage comparison graphs.

2. Experimental Setup

The algorithms were implemented in Python. Execution time was measured with time.perf_counter() and peak traced memory with the tracemalloc module. Each algorithm was tested with input sizes of 100, 500, 1000 and 5000. Every benchmark was repeated five times, and the average execution time and average peak traced memory were recorded.

3. Solution (Code)

3.1 Fractional Knapsack

Items are sorted by value-to-weight ratio and taken greedily; the last item may be taken partially.

```
def fractional_knapsack(items, capacity):
    """
    Select items to maximize total value.
    Fractions of items are allowed.
    """

    if capacity < 0:
        raise ValueError("Capacity cannot be negative.")

    if any(
        item["weight"] <= 0 or item["value"] < 0
        for item in items
    ):
        raise ValueError(
            "Weights must be positive and values non-negative."
        )

    # Sort by value-to-weight ratio in descending order.
```

```
    sorted_items = sorted(
        items,
        key=lambda item: item["value"] / item["weight"],
        reverse=True
    )

    total_value = 0.0
    total_weight = 0.0
    remaining = capacity
    selected = []

    for item in sorted_items:
        if remaining <= 0:
            break

        weight = item["weight"]
        value = item["value"]

        if weight <= remaining:
            fraction = 1.0
        else:
            fraction = remaining / weight

        allocated_weight = weight * fraction
        allocated_value = value * fraction

        selected.append({
            "name": item["name"],
            "fraction": fraction,
            "weight": allocated_weight,
            "value": allocated_value
        })

        total_weight += allocated_weight
        total_value += allocated_value
        remaining -= allocated_weight

    return total_value, total_weight, selected
```

3.2 Activity Selection

Activities are sorted by finishing time; an activity is selected if it starts at or after the finish time of the previously selected one.

```
def activity_selection(activities):
    """
    Select the maximum number of non-overlapping activities.
    Each activity has a name, start time, and finish time.
    """

    if any(
        activity["start"] > activity["finish"]
        for activity in activities
    ):
        raise ValueError("Start time cannot exceed finish time.")

    # Select activities in increasing order of finish time.
    sorted_activities = sorted(
        activities,
        key=lambda activity: activity["finish"]
    )

    selected = []
    last_finish = None

    for activity in sorted_activities:
        if (
            last_finish is None
            or activity["start"] >= last_finish
        ):
            selected.append(activity)
            last_finish = activity["finish"]

    return selected
```

3.3 Huffman Coding

Character frequencies are counted, a Huffman tree is built with a min-heap (priority queue), and binary codes are generated by traversing the tree (left edge = 0, right edge = 1).

```python
import heapq
from collections import Counter
from itertools import count


class Node:
    def __init__(self, char, freq, left=None, right=None):
        self.char = char
        self.freq = freq
        self.left = left
        self.right = right


def build_huffman_tree(text):
    if not text:
        raise ValueError("Input text cannot be empty.")

    frequencies = Counter(text)
    unique_id = count()

    heap = [
        (freq, next(unique_id), Node(char, freq))
        for char, freq in frequencies.items()
    ]

    heapq.heapify(heap)

    while len(heap) > 1:
        left_freq, _, left = heapq.heappop(heap)
        right_freq, _, right = heapq.heappop(heap)

        merged = Node(
            None,
            left_freq + right_freq,
            left,
            right
        )

        heapq.heappush(
            heap,
```

```
            (merged.freq, next(unique_id), merged)
        )

    return heap[0][2]


def generate_codes(node, prefix="", codebook=None):
    if codebook is None:
        codebook = {}

    if node.char is not None:
        codebook[node.char] = prefix or "0"
        return codebook

    generate_codes(node.left, prefix + "0", codebook)
    generate_codes(node.right, prefix + "1", codebook)

    return codebook


def huffman_encode(text):
    if not text:
        raise ValueError("Input text cannot be empty.")

    root = build_huffman_tree(text)
    codebook = generate_codes(root)

    encoded = "".join(codebook[char] for char in text)

    original_bits = len(text) * 8
    encoded_bits = len(encoded)

    compression_ratio = (
        encoded_bits / original_bits
        if original_bits > 0
        else 0
    )

    space_saved = (
        (1 - compression_ratio) * 100
```

```
        if original_bits > 0
        else 0
    )

    return {
        "root": root,
        "frequencies": dict(Counter(text)),
        "codes": codebook,
        "encoded": encoded,
        "original_bits": original_bits,
        "encoded_bits": encoded_bits,
        "compression_ratio": compression_ratio,
        "space_saved_percent": space_saved
    }
```

3.4 Benchmarking (time and memory)

```
import time
import tracemalloc


def measure_performance(function, *args, repeats=5):
    """
    Measure execution time and peak traced Python memory.

    Returns:
        average execution time in seconds
        peak traced memory in KB
    """

    execution_times = []
    peak_memory_kb = []

    for _ in range(repeats):
        tracemalloc.start()

        start_time = time.perf_counter()

        function(*args)

        elapsed = time.perf_counter() - start_time
```

```
        current_memory, peak_memory = tracemalloc.get_traced_memory()

        tracemalloc.stop()

        execution_times.append(elapsed)
        peak_memory_kb.append(peak_memory / 1024)

    average_time = sum(execution_times) / repeats
    average_memory = sum(peak_memory_kb) / repeats

    return average_time, average_memory
```

3.5 Visualization code

```
import os
import matplotlib.pyplot as plt
import networkx as nx

from algorithms.fractional_knapsack import fractional_knapsack
from algorithms.activity_selection import activity_selection
from algorithms.huffman_coding import huffman_encode

os.makedirs("graphs", exist_ok=True)


# 1. Resource allocation chart: Fractional Knapsack
items = [
    {"name": "Resource A", "weight": 10, "value": 60},
    {"name": "Resource B", "weight": 20, "value": 100},
    {"name": "Resource C", "weight": 30, "value": 120},
]
capacity = 30

total_value, total_weight, selected = fractional_knapsack(
    items, capacity
)

names = [item["name"] for item in selected]
allocated_weights = [item["weight"] for item in selected]
```

```
plt.figure(figsize=(8, 5))
plt.bar(names, allocated_weights)
plt.xlabel("Selected Resources")
plt.ylabel("Allocated Weight")
plt.title(
    f"Fractional Knapsack Resource Allocation "
    f"(Capacity = {capacity})"
)
plt.tight_layout()
plt.savefig("graphs/resource_allocation.png", dpi=150)
plt.close()


# 2. Activity selection timeline
activities = [
    {"name": "A1", "start": 1, "finish": 4},
    {"name": "A2", "start": 3, "finish": 5},
    {"name": "A3", "start": 0, "finish": 6},
    {"name": "A4", "start": 5, "finish": 7},
    {"name": "A5", "start": 8, "finish": 9},
    {"name": "A6", "start": 5, "finish": 9},
]

selected_activities = activity_selection(activities)

plt.figure(figsize=(10, 5))

for row, activity in enumerate(selected_activities):
    start = activity["start"]
    duration = activity["finish"] - start

    plt.barh(
        row,
        duration,
        left=start,
        height=0.5
    )

    plt.text(
```

```
        start + duration / 2,
        row,
        activity["name"],
        ha="center",
        va="center"
    )

plt.yticks(
    range(len(selected_activities)),
    [a["name"] for a in selected_activities]
)
plt.xlabel("Time")
plt.ylabel("Selected Activities")
plt.title("Activity Selection Timeline")
plt.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.savefig("graphs/activity_timeline.png", dpi=150)
plt.close()


# 3. Huffman tree representation
text = "greedyalgorithms"
huffman_result = huffman_encode(text)
root = huffman_result["root"]

graph = nx.DiGraph()
labels = {}
counter = [0]


def add_tree_nodes(node, parent=None, edge_label=""):
    node_id = counter[0]
    counter[0] += 1

    if node.char is None:
        label = f"Freq: {node.freq}"
    else:
        label = f"{repr(node.char)}\\nFreq: {node.freq}"

    graph.add_node(node_id)
```

```
    labels[node_id] = label

    if parent is not None:
        graph.add_edge(parent, node_id, label=edge_label)

    if node.left is not None:
        add_tree_nodes(node.left, node_id, "0")

    if node.right is not None:
        add_tree_nodes(node.right, node_id, "1")


add_tree_nodes(root)

# Assign each tree node a hierarchical position.
positions = {}


def assign_positions(node, depth=0, x_counter=None):
    if x_counter is None:
        x_counter = [0]

    node_id = next(
        n for n in graph.nodes
        if labels[n] == (
            f"Freq: {node.freq}"
            if node.char is None
            else f"{repr(node.char)}\\nFreq: {node.freq}"
        )
        and n not in positions
    )

    if node.left is None and node.right is None:
        x = x_counter[0]
        x_counter[0] += 1
    else:
        child_x = []

        if node.left is not None:
            child_x.append(
```

```
            assign_positions(node.left, depth + 1, x_counter)
        )

    if node.right is not None:
        child_x.append(
            assign_positions(node.right, depth + 1, x_counter)
        )

    x = sum(child_x) / len(child_x)

  positions[node_id] = (x, -depth)
  return x


# Use a direct tree traversal with stable node IDs.
positions.clear()
tree_counter = [0]


def layout_tree(node, depth=0, x_counter=None):
  if x_counter is None:
    x_counter = [0]

  node_id = tree_counter[0]
  tree_counter[0] += 1

  if node.left is None and node.right is None:
    x = x_counter[0]
    x_counter[0] += 1
  else:
    child_x = []

    if node.left is not None:
      child_x.append(layout_tree(node.left, depth + 1, x_counter))

    if node.right is not None:
      child_x.append(layout_tree(node.right, depth + 1, x_counter))

    x = sum(child_x) / len(child_x)
```

```
    positions[node_id] = (x, -depth)
    return x


positions.clear()
layout_tree(root)

plt.figure(figsize=(14, 8))
nx.draw(
    graph,
    positions,
    labels=labels,
    with_labels=True,
    node_size=1800,
    font_size=8,
    arrows=False
)

edge_labels = nx.get_edge_attributes(graph, "label")
nx.draw_networkx_edge_labels(
    graph,
    positions,
    edge_labels=edge_labels,
    font_size=9
)

plt.title("Huffman Coding Tree (0 = Left, 1 = Right)")
plt.axis("off")
plt.tight_layout()
plt.savefig("graphs/huffman_tree.png", dpi=150)
plt.close()

print("Saved all three algorithm visualizations:")
print("graphs/resource_allocation.png")
print("graphs/activity_timeline.png")
print("graphs/huffman_tree.png")
```

4. Theoretical Complexity

Algorithm	Theoretical Time Complexity	Auxiliary Space Complexity

Fractional Knapsack	O(n log n)	O(n)
Activity Selection	O(n log n)	O(n)
Huffman Coding	O(L + k log k)	O(k) for tree and supporting structures; O(L) additional for encoded output

Here, n is the number of items or activities, L is the length of the input text, and k is the number of distinct characters. Fractional Knapsack and Activity Selection are dominated by sorting, giving O(n log n) time. Huffman Coding needs O(L) for frequency counting and encoding and O(k log k) to build the tree, giving O(L + k log k).

5. Output Screenshots

5.1 Fractional Knapsack – Resource Allocation Chart

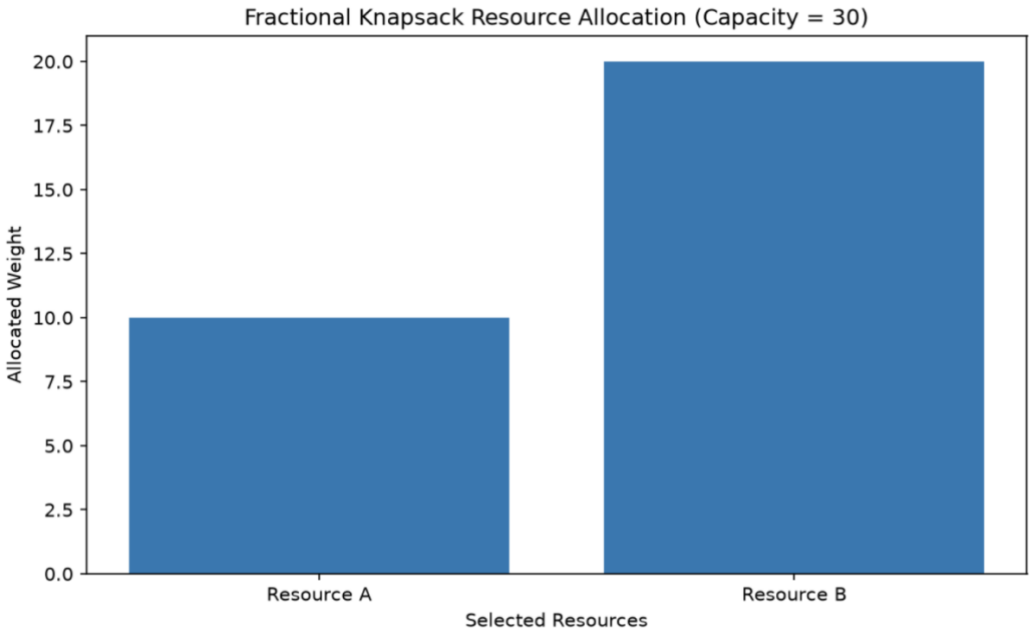


Figure 1. Fractional Knapsack resource allocation (capacity = 30).

The available capacity is allocated among the selected resources according to their value-to-weight ratios; an item may be taken partially when the remaining capacity cannot hold it entirely.

5.2 Activity Selection – Timeline

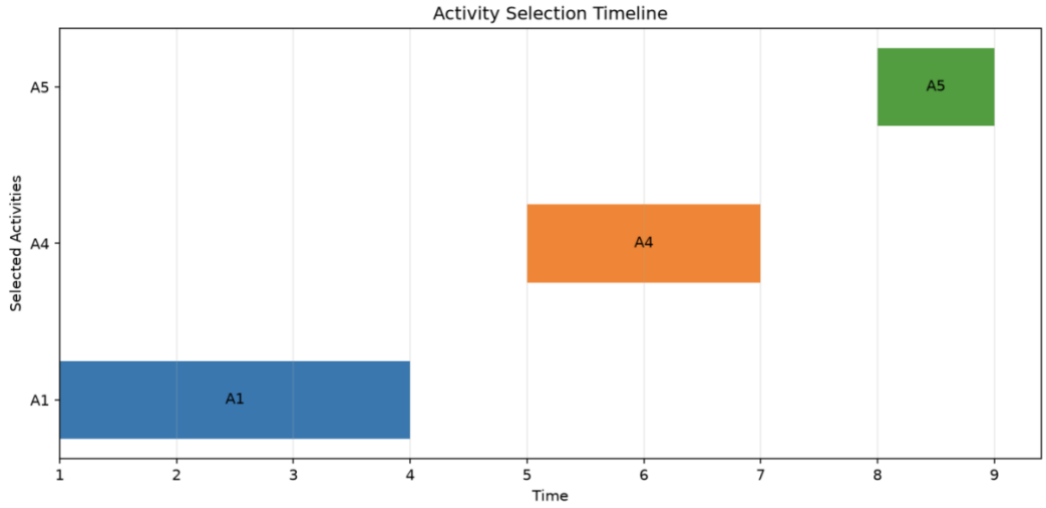


Figure 2. Timeline of the activities selected (A1, A4, A5).

The algorithm sorts activities by finishing time and selects those that do not overlap with previously selected activities.

5.3 Huffman Coding – Tree Representation

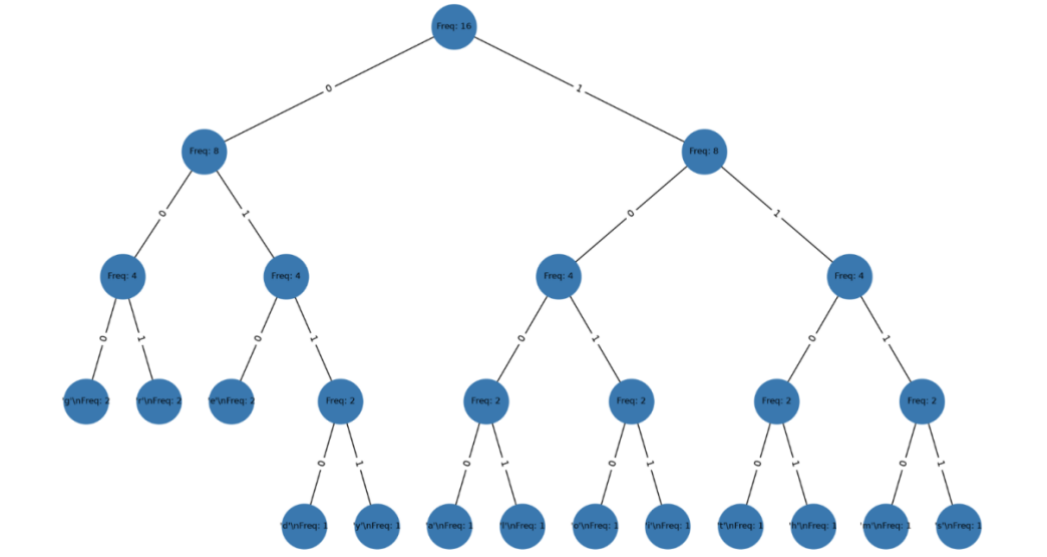


Figure 3. Huffman tree (left edge = 0, right edge = 1).

Each leaf represents a character and each internal node a combined frequency. The left and right edges correspond to the binary digits 0 and 1.

5.4 Execution-Time Comparison

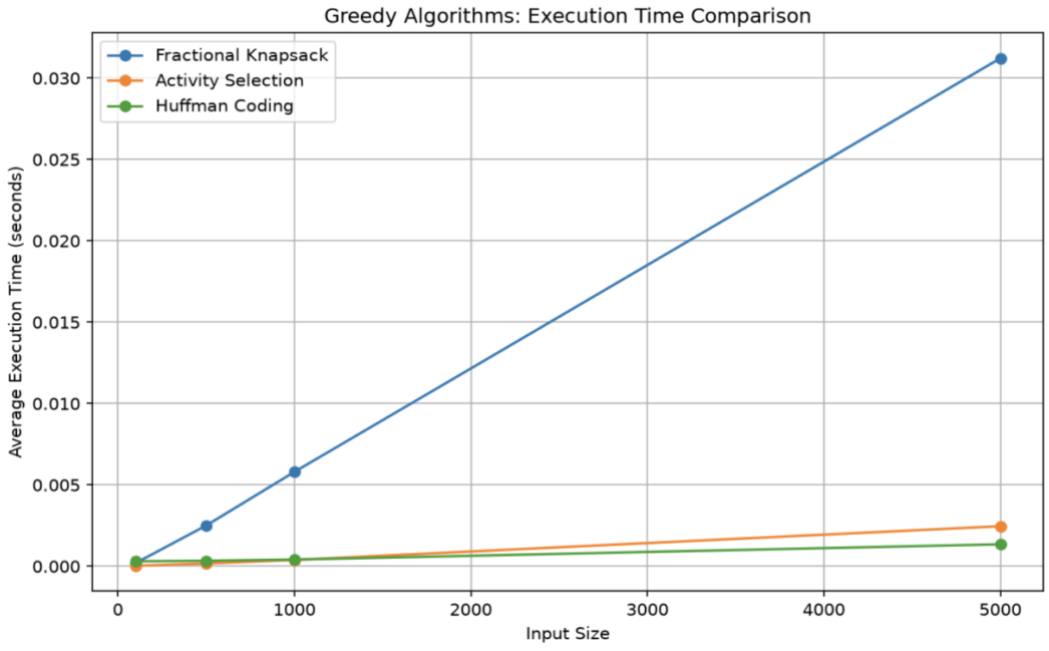


Figure 4. Average execution time versus input size (100, 500, 1000, 5000).

5.5 Memory-Usage Comparison

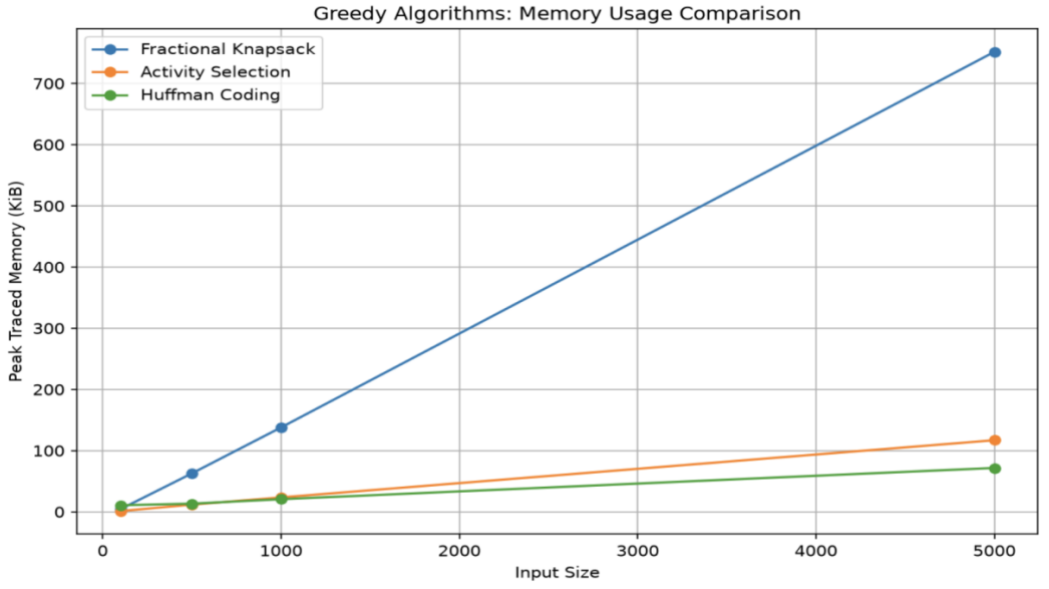


Figure 5. Peak traced memory (tracemalloc) versus input size.

6. Analysis and Observations

Fractional Knapsack shows the highest execution time and memory usage as the input grows (about 0.031 s and roughly 750 KiB at n = 5000 in the graphs), consistent with its O(n log n) sort over a list of tuples. Activity Selection also sorts, so it is O(n log n) too, but it handles smaller records and stays well below Fractional Knapsack in both time and memory.

Huffman Coding depends on the text length L and the number of distinct characters k. The test generator uses only 26 lowercase letters, so $k \leq 26$ and tree building stays cheap compared with

processing the text; its time grows roughly linearly with L and it uses the least memory at the largest input size.

The memory graph reports Python-traced allocations rather than the memory of the whole Python process. Actual measurements vary with system load, Python runtime overhead and implementation details, so the results should be compared with the theoretical complexities allowing for that variation.

7. Conclusion

The experiment implemented three greedy algorithms and measured their execution time and memory for increasing input sizes. The visualizations (allocation chart, activity timeline and Huffman tree) explain how each algorithm works, and the performance graphs show that the observed growth agrees with the theoretical complexities for the tested input sizes.

8. GitHub Link

GitHub Repository: https://github.com/Dipanshuohlyan84/DAA-lab3